

AIKR PROJECT: STUDYCHINGU

Name: Kota Aasrita

Roll no.: S20240030382

Name: Koredla Sai Sevitha

Roll no.: S20240030381

Name: Velicharla Madhu Sravani

Roll no.: S20240030407

I.Problem Statement:

Students often struggle to study effectively from raw notes and textbooks, especially when studying for the exam the night before. The core challenges are: passively re-reading material without understanding, having no structured study plan, losing track of topics across and within subjects, and lacking immediate feedback when a concept is misunderstood.

Existing tools such as general chatbots or flashcard apps address one of these problems in isolation. The available sites providing similar utilities either have a website that is a pain to navigate, or are barred by credits and subscriptions. There is no unified system that can read a student's own notes and simultaneously answer doubts, generate quizzes, create flashcards, identify weak topics, and build a personalized study timetable all from a single interface while being free of cost.

This is why our AI agent **StudyChingu** is suitable here because the problem requires perceiving inputs (uploaded notes, quiz results, user preferences), reasoning about them (which topics are weak, how much time to allocate), and acting to achieve a goal (helping the student prepare for exams). A simple chatbot cannot plan, remember state across sessions, track your progress automatically, set valid deadlines or make scheduling decisions.

- **Environment:** uploaded study notes (PDFs/text), student inputs, quiz history in progress.json
- **Agent type:** Goal-based agent
- **Percepts:** uploaded files, typed questions, quiz answers, subject inputs, free time slots
- **Actions:** generate notes, answer doubts, quiz student, create flashcards, build timetable

II.Agent Architecture:

StudyChingu is a goal-based agent implemented as a Python application. Each module handles a distinct part of the agent's decision cycle.

- **app.py**: Main Streamlit UI; perception layer that routes user inputs to the correct module
- **rag pipeline**: PDF ingestion, keyword-based chunking, semantic retrieval
- **prompts.py**: System prompts for all five LLM-powered features
- **Quiz_engine.py**: From-scratch quiz scoring, streak tracking, and wrong-answer logging
- **progress.py**: From-scratch JSON-based knowledge base that stores all session history
- **rule_engine.py**: Forward chaining inference engine; production rules fire on quiz/deadline facts
- **timetable_agent.py**: Hybrid CSP + LLM scheduler

Planned Work flow:

1. User uploads a PDF → PyMuPDF extracts text → split into 800-character chunks.
2. The user asks a question or requests a quiz → keyword scoring retrieves the top-3 most relevant chunks → chunks sent to Groq (Llama 3.3-70b) as context.
3. Quiz results → quiz_engine.py scores and logs wrong answers → progress.py saves session to progress.json.
4. Timetable request → rule_engine.py reads progress.json, fires production rules, asserts facts into working memory → timetable_agent.py feeds working memory into CSP scheduler → Groq personalises the task descriptions

III.AI Techniques used:

Knowledge Representation

progress.json acts as the agent's persistent knowledge base. It stores quiz sessions, cumulative wrong-answer counts per topic, and timetable history. All read/write logic was written from scratch in progress.py using Python's built-in json and os modules — no database library used.

Rule-Based Reasoning (Forward Chaining Engine)

rule_engine.py implements a forward chaining inference engine. \ Production rules are defined as condition → action pairs operating on a WorkingMemory object.

- rule_critical_exam: IF exam in ≤ 2 days AND avg score < 60% THEN priority = CRITICAL
- rule_weak_topic: IF quiz avg < 60% THEN flag as weak AND multiply time allocation by 1.5
- rule_user_override: IF user says 'weak' BUT quiz avg $\geq 80\%$ THEN agent overrides self-report
- rule_deadline_urgency: IF deadline ≤ 3 days THEN time_multiplier = 1.8–2.5
- rule_cascade_critical: IF CRITICAL AND WEAK (chained rule) THEN status = EMERGENCY, multiplier = 3.0

Rule chaining is demonstrated by rule_cascade_critical, which only fires if two earlier rules have already asserted facts into working memory.

Constraint Satisfaction (CSP Timetable Scheduler)

timetable_agent.py implements a greedy CSP scheduler in build_time_blocks().

Constraints enforced by the algorithm (not the LLM):

- C1: Study blocks never exceed user's declared free time slots
- C2: No continuous study block exceeds 50 minutes — breaks inserted automatically
- C3: Subjects scheduled in strict priority order from rule engine output
- C4: Subject time capped at 60% of total available time to prevent imbalance
- C5: Subjects split across multiple slots if a single slot cannot fit them

RAG — Retrieval Augmented Generation

We implemented a lightweight RAG pipeline without LlamaIndex. PyMuPDF (fitz) extracts text from uploaded PDFs. `split_chunks()` breaks text into 800-character sentence-aware chunks. `find_relevant()` scores chunks by keyword overlap with the user's question and returns the top-3 most relevant chunks as context for the LLM.

Large Language Model Plug in (Groq (Llama 3.3-70b-versatile))

The Groq API is used for five features: note making, doubt answering, quiz generation, flashcard generation, and timetable task personalisation. For timetable generation the LLM is restricted to writing task descriptions only; all scheduling decisions (times, durations, subject order) are made by the algorithm.

IV. System Design:

Component	Technology	Purpose
Frontend / UI	Streamlit	Web interface — sidebar navigation, file upload, charts
LLM API	Groq (Llama 3.3-70b)	Natural language generation for all 5 AI features
PDF parsing	PyMuPDF (fitz)	Extract text from uploaded PDFs
Data store	JSON file (progress.json)	Persistent knowledge base
Visualisation	Plotly	Interactive score charts and progress dashboard
Rule engine	Custom Python class	Forward chaining inference
CSP scheduler	Custom Python algorithm	Time-block placement

StudyAgentAIKR/

style.css

source_code/

app.py

prompts.py

quiz_engine.py

rule_engine.py

timetable_agent.py

progress.py

data/progress.json

requirements.txt

[README.md](#)

IV.Implimentation Details:

Quiz Engine (quiz_engine.py)

The QuizEngine class manages the full lifecycle of a quiz session. It stores questions as a list of dicts, tracks the current question index, score, streak, and best streak.

submit_answer() compares the student's choice to the correct answer, updates state, and logs wrong answers with their topic labels. get_summary() returns a complete dict used by progress.py to persist the session.

Forward Chaining Rule Engine (rule_engine.py)

WorkingMemory stores facts as a Python dict (key → value). Each rule function accepts the working memory and a subject, checks its condition against progress data, and calls wm.assert_fact() if the condition is met. run_inference() iterates over all subjects and fires all six rules in order. Chained rules (rule_cascade_critical) read facts asserted by earlier rules.

CSP Timetable Scheduler (timetable_agent.py)

build_time_blocks() parses user-defined free slots into datetime objects. It expands subjects into 50-minute chunks based on their rule-engine-allocated time. A greedy placement loop fills each slot, inserting 10-minute breaks after every 50 minutes of continuous study. Subject blocks are split across multiple slots if needed. The LLM (Phase 2) receives only the completed block schedule and writes one task description per block; it cannot alter times or durations.

RAG Pipeline (app.py)

split_chunks() splits extracted text on sentence boundaries, grouping sentences until an 800-character threshold is reached. find_relevant() converts the user's question and each chunk into keyword sets, scores chunks by intersection size, and returns the top-3 chunks. This is a simple TF-style retrieval approach that avoids external embedding APIs and works reliably on student notes.

Progress Tracker (progress.py)

All data is stored in a single JSON file. _load() reads it (returning a blank structure if the file doesn't exist). _save() writes it back. save_session() appends a new session dict and increments cumulative wrong-topic counts. get_weak_topics() sorts the topic dict by count and returns the top N, this feeds directly into the rule engine and the timetable generator.

Missed Task Rescheduler (progress.py)

The function `get_missed_tasks()` returns any incomplete tasks, its companion function, `suggest_reschedule_slots()` scores the next seven days using urgency, free capacity, and subject overload penalty. `reschedule_task()` moves the task to the chosen date in `progress.json`. Where urgency is higher for closer days, an overload penalty is applied if the same subject already appears heavily that day.

VI.Results and Testing:

All five core features of the Study Agent were tested using CCN (Computer Communication Networks) Unit 4 lecture slides as the uploaded study material. The AIKR subject was used for timetable testing. Every feature was verified end to end through the Streamlit web interface.

Feature	Test performed	Outcome
■ Note Maker	Uploaded CCN Unit 4 slides PDF, clicked Generate Notes	PASS — Clean structured notes generated with headings, subheadings and bullet points directly from the slide content. Output was accurate and well-organised.
■ Flashcards	Clicked Generate Flashcards on the same CCN PDF	PASS — 8 key concept flashcard pairs generated. Cards were accurate, relevant to the material, and flippable in the UI.
■ Quiz	Generated quiz on CCN Unit 4, attempted all 15 questions	PASS — 15 MCQs generated from uploaded notes. Scoring, streak tracking and wrong-answer logging all functioned correctly.
■ Progress & Dashboard	Checked Progress page and Dashboard after completing quiz	PASS — Bar chart updated with subject score. Weak topics detected and displayed. Dashboard showed correct session count, streak and agent recommendation.
■ Timetable	Added AIKR as subject, set free time slot, generated timetable	PASS — Rule engine fired correctly, CSP scheduler allocated time block, Groq personalised the task description. Rule firing trace displayed in UI.

Edge Case Testing

Large PDF context limit (195 pages): A 195-page textbook was uploaded. Concepts in the later half were never embedded due to the chunk limit and top-3 retrieval. When a doubt was asked about one of those topics, the system responded "This topic is not covered in your uploaded material", even though it was in the PDF. The student would miss the concept without realising the system simply never read that far.

PDF with diagrams and equations: A Chemistry/Physics PDF consisting mostly of diagrams and equations was uploaded. PyMuPDF only extracts the text layer, diagrams are invisible and equations render as broken symbols. The notes appeared clean and well-structured so the student would trust them, but entire visual concepts were silently missing.

All subjects weak, small time window: Multiple subjects were all marked weak with only a 60-minute slot available. The rule engine fired on all of them and applied time multipliers, but since total time was fixed the CSP compressed every subject equally into tiny slots. The multipliers became meaningless, a weak subject getting 20 minutes is no different from a strong one.

Insufficient material for quiz: Half a page of notes was uploaded and Generate Quiz was clicked. With only 15 questions forced by the prompt, the LLM ran out of distinct content and began rephrasing the same concept in different wordings to fill the quota.

VII.Limitations:

- RAG retrieval uses keyword overlap, it does not use vector embeddings, so semantically similar but differently worded sentences may not be retrieved.
- progress.json is a flat file — it will become slow with very large numbers of sessions and would need to be replaced with a database for a production system.
- The timetable agent assumes the student's free slots are accurate, it cannot detect conflicts with external calendars.
- The quiz generator occasionally produces fewer than the requested number of questions if the uploaded material is very short or lacks factual content.
- The system currently supports only English-language notes.
- Flashcard and quiz quality depends on the quality and length of uploaded material, very short notes produce generic questions.
- There is no user authentication, progress.json is shared on the same machine

VIII.AI Declaration:

Claude (Anthropic)

- **Purpose:** Project planning, initial code scaffolding, debugging assistance
- **AI-generated:** Initial app.py structure and session state boilerplate; suggestions for prompt wording
- **Self-written:** All from-scratch modules: quiz_engine.py, rule_engine.py, progress.py, timetable_agent.py (CSP logic); all system prompts refined by team; full UI layout

Groq / Llama 3.3

- **Purpose:** Runtime AI feature — used inside the application itself
- **AI-generated:** Note summaries, quiz questions, flashcards, timetable task descriptions (at runtime)

IX.References:

[1] Russell, S. J., & Norvig, P. (2022). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. [Agent architecture, forward chaining, CSP — Chapters 2, 6, 7]

[2] AIKR S2026 Course Lecture Slides — Unit 2: Informed Search, Unit 3: Constraint Satisfaction Problems, Unit 4: Knowledge Representation, Unit 5: Reasoning and Forward Chaining, Unit 6: Planning.

[3] Streamlit Inc. (2024). Streamlit Documentation. <https://docs.streamlit.io>

[4] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*.

[5] A-R007. (2024). Multi-Agent-Study-Assistant — A multi-agent system for student learning. GitHub. <https://github.com/A-R007/Multi-Agent-Study-Assistant>

[6] sarim2000. (2024). PennywiseAI Tracker — AI-powered personal finance tracker using agentic architecture. GitHub. <https://github.com/sarim2000/pennywiseai-tracker>

[7] HarimxChoi. (2024). LangGraph Travel Agent — A goal-based travel planning agent built with LangGraph. GitHub. <https://github.com/HarimxChoi/langgraph-travel-agent>

[8] atick-faisal. (2024). Expense Tracker Android — Rule-based expense categorisation and tracking system. GitHub. <https://github.com/atick-faisal/Expense-Tracker-Android>